



Reporte Técnico

Proyecto final

Universidad de Guadalajara
Centro Universitario de Ciencias Exactas e Ingenierías
División de Tecnologías para la Integración Ciber-Humana

Calendario 2026A
IL360 – Programación Paralela y Concurrente
NRC: 236064

Jonathan Martin Melendez Mendoza
218543729
Ingeniería en Computación (ICOM)
27/05/26

Código desarrollado

Repositorio de código: <https://github.com/qJona65p/PPyC---OpenMP>

Especificaciones del hardware de prueba

CPU

- Modelo: Ryzen 7 5800X
- Núcleos físicos: 8 núcleos
- Núcleos lógicos: 16 hilos

Memoria

- L1d: 256 KiB (8 instances)
- L1i: 256 KiB (8 instances)
- L2: 4 MiB (8 instances)
- L3: 32 MiB (1 instance)
- RAM: 32 GB

Metodología

Prompt inicial

Generar un programa en C++ puramente secuencial que realice las siguientes dos tareas:

- Tarea A: Generar una imagen 8K del Conjunto de Mandelbrot.
- Tarea B: Aplicar un filtro de convolución 2D pesado, un desenfoque Gaussiano de radio amplio sobre la imagen generada.

Prompt de paralelización

Ahora, haciendo uso de OpenMP implementa paralelismo en las tareas.

Optimizaciones en el código de la IA

Inicialmente usaba un numero muy bajo de iteraciones para la generación de mandelbrot, por lo que fue un valor aumentado. Así como la optimización del blur, cambiando el kernel que cambio de $\sigma = radius / 2.0$ a $\sigma = radius / 3.0$.

Comandos utilizados

Compilación base:

```
g++ -O2 -std=c++17 -fopenmp -o mandelbrot_omp mandelbrot_omp.cpp
```

Compilación vectorizada:

```
g++ -O3 -std=c++17 -fopenmp -march=native -ftree-vectorize -o mandelbrot_omp mandelbrot_omp.cpp
```

Monitoreo de cache:

```
perf stat -e cache-misses,cache-references ./mandelbrot_omp
```

Obtención de resultados

El proyecto se fue construyendo iterativamente sobre el programa paralelizado, cada función nueva y tests realizados están en cada commit del repositorio, se tomó cada salida de el programa y se fue documentando en archivos Markdown.

Gráficas y resultados

Comparación de schedulers

Tiempo de ejecución por scheduler y chunk size

Chunk Size	Dynamic (s)	Static (s)	Guided (s)
1	1.19301	1.24539	1.37554
2	1.21258	1.25504	1.40224
4	1.15799	1.18727	1.37467
8	1.15336	1.19625	1.42052
16	1.17552	1.26987	1.46039
32	1.18561	1.27342	1.41108
64	1.34727	1.41294	1.53674
128	1.39833	1.38828	1.71127

Mejor chunk size por scheduler

Scheduler	Mejor Chunk Size	Mejor tiempo (s)
Dynamic	8	1.15336
Static	4	1.18727
Guided	4	1.37467

Rendimiento relativo vs mejor (dynamic, chunk = 8)

Scheduler	Time (s)	Slowdown
Dynamic	1.15336	Baseline
Static	1.18727	+2.9%
Guided	1.37467	+19.2%

Podemos observar que el scheduler(dynamic) fue el sobresaliente en todas las situaciones, esto es porque la estructura de los fractales favorece una ejecución dinámica, los nodos mas externos requieren un tiempo de calculo mayor a los internos, provocando un desbalance que los schedulers *static* y *guided* no pueden manejar fácilmente. [1]

Histogramas

Comparación entre las implementaciones del histograma

Implementación	Tiempo en exec (s)	Colores únicos	Speedup vs Critical
Critical section	6.88902	1510	1.00×
Local variables	0.023033	1510	299.09×

Top 5 colores mas frecuentes

Rank	R	G	B	HEX	Pixels
1	0	0	0	#000000	5,963,229
2	254	57	70	#FE3946	3,649,794
3	254	55	72	#FE3748	2,769,433
4	254	56	71	#FE3847	2,468,499
5	254	60	67	#FE3C43	2,451,282

False sharing

No hay ninguna señal de que ocurra false sharing en el programa, un test que fue realizado fue ejecutar el programa con 1 solo hilo y 16 hilos, para contrastar el porcentaje de cache misses:

Ejecución con 1 solo hilo:

87 795 507	cache-misses	#	1.35% of all cache refs
6 490 137 188	cache-references		

Ejecución con 16 hilos:

72 682 210	cache-misses	#	1.17% of all cache refs
6 228 438 108	cache-references		

Por si mismo los cache misses no son un indicativo de *false sharing* pero pueden hacerlo notar si aumentan al tener paralelismo por que mostraría que los hilos están intentando acceder a la misma sección provocando errores. [2]

Forzar vectorización usando SPMD

Para lograr esto el único cambio necesario fue utilizar la sentencia `#pragma omp simd reduction(+:vr,vg,vb)` a los bucles mas internos del filtro, pero esto incluso pareció entorpecer la ejecución, porque el tiempo total aumentó.

Usando la *flag -fopt-info-vec-optimized* al momento de compilar podemos ver que efectivamente se forzó la vectorización como se puede ver en el archivo *vectorization_out.txt* del repositorio.

La causa de la ralentización sería la propia estructura del código, puesto que optimizarlo realmente para funcionar con SPMD requeriría una reestructuración completa y compleja del código [3], la forma en la que esta implementado actualmente solo vectoriza los ciclos que iteran el kernel, siendo 31 iteraciones, mientras que la optimización ideal iteraría sobre los 7680 píxeles del ancho de la imagen.

Afinidad

Comparación de configuraciones de Afinidad

Configs	Threads	Proc Bind	Places	Task A (s)	Task B (s)	Total Time (s)	Cache Misses	Cache Refs	Miss Rate
Default (no flags)	16	Default	Default	1.74231	1.17419	5.21041	146,991,640	5,854,898,201	2.51%
Config 1	16	spread	cores	1.60743	1.13877	5.05193	141,804,411	6,017,438,968	2.36%
Config 2	8	spread	cores	1.49319	1.15814	4.84301	129,037,277	6,199,677,332	2.08%
Config 3	16	closed	threads	1.53771	0.94772	4.58780	106,943,812	6,248,842,853	1.71%
Config 4	8	closed	cores	1.65799	1.15390	4.96444	113,640,014	6,219,672,619	1.83%

Mejora respecto al default

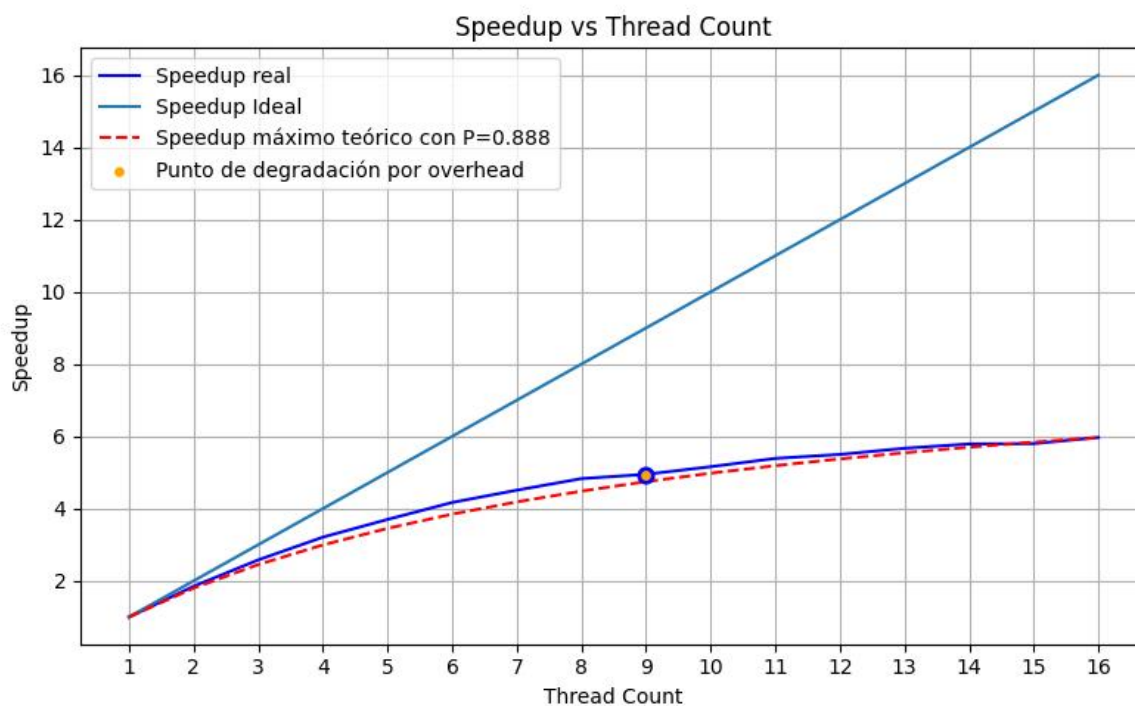
Configuración	Mejora de tiempo total	Reducción de Cache Miss
Config 1 (16, spread, cores)	3.0%	3.5%
Config 2 (8, spread, cores)	7.0%	12.2%
Config 3 (16, closed, threads)	11.9%	27.2%
Config 4 (8, closed, cores)	4.7%	22.7%

Observaciones

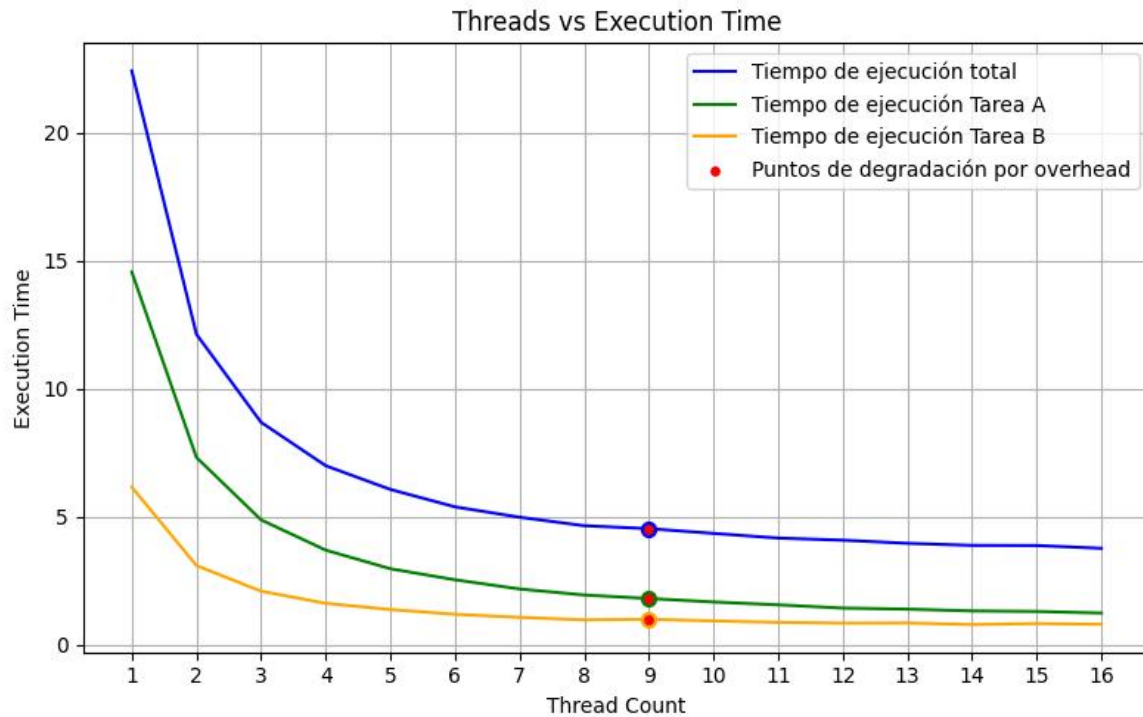
Observación	Flags (Proc_Bind, Places, Num_threads)
Ejecución más rápida	closed, threads, 16
Menos cache misses	closed, threads, 16
Generación Mandelbrot más rápida (Task A)	spread, cores, 8
Filtro Gaussian Blur más rápido (Task B)	closed, threads, 16

Tiempo de Ejecución vs Numero de hilos vs Speedup

Threads	Task A (s)	Task B (s)	Tiempo total (s)	Speedup
1	14.55100	6.14614	22.40440	1.00×
2	7.30743	3.08553	12.11910	1.85×
3	4.86816	2.08034	8.67843	2.58×
4	3.68461	1.60732	6.98253	3.21×
5	2.95771	1.36441	6.05611	3.70×
6	2.52630	1.17882	5.37496	4.17×
7	2.16670	1.05869	4.96944	4.51×
8	1.93162	0.95921	4.63854	4.83×
9	1.79413	0.98691	4.52371	4.95×
10	1.65906	0.91921	4.33845	5.16×
11	1.54750	0.86451	4.15566	5.39×
12	1.42096	0.83276	4.07319	5.50×
13	1.37945	0.83895	3.95097	5.67×
14	1.31011	0.77777	3.87049	5.79×
15	1.28887	0.81663	3.86496	5.80×
16	1.22814	0.78671	3.75561	5.97×



Gráfica 1: Speedup vs Hilos utilizados.



Gráfica 2: Tiempo de ejecución vs hilos utilizados.

Análisis del rendimiento

Fracción paralelizable y speedup teórico

Utilizando la ley de Amdahl y los resultados obtenidos podemos estimar la fracción paralelizable:

$$S(N) = \frac{1}{(1-P) + \frac{P}{N}} \quad \text{Entonces: } 5.97 = \frac{1}{(1-P) + \frac{P}{16}}$$

$$\text{Simplificando: } 0.1675 = (1-P) + 0.0625 P$$

$$0.1675 = 1 - 0.9375 P$$

$$0.9375 P = 0.8325$$

$$P = 0.888$$

Por lo que cerca de un 89% del código es paralelizable, teniendo esto en cuenta podemos calcular el speedup teórico máximo de la siguiente forma:

$$S_{max} = \frac{1}{1-P} \quad \text{Entonces: } S_{max} = \frac{1}{1-0.888} = \frac{1}{0.112} = 8.93$$

$$S_{max} = 8.93$$

De un speedup teórico máximo de 8.93x conseguimos con 16 hilos un speedup de 5.97x alcanzando un 63.2% de ese speedup máximo.

Degradación por overhead

Podemos notar que el speedup real aumenta a buen ritmo hasta llegar a 8 hilos, momento en el que se comienza a utilizar hyperthreading para alojar 2 hilos por núcleo, a partir de 9+ hilos las ganancias en speedup son menores a 0.2x respecto al anterior, además de que la Tarea B, empeora su tiempo de ejecución de 0.95921 a 0.98691 segundos a pesar del hilo adicional.

Habiendo empeorado el tiempo de ejecución de la Tarea B y tener ganancias peores podemos concluir que en ese momento el overhead del sistema comienza a reducir el rendimiento adicional por cada hilo agregado.

Conclusiones

En este proyecto desarrollamos e iteramos un generador de fractales de Mandelbrot a resolución 8K. Partimos de una implementación completamente secuencial y fuimos avanzando hasta conseguir una versión paralela con OpenMP, incorporando múltiples optimizaciones y mejoras funcionales.

La versión paralela alcanzó un speedup máximo de 5.97x usando 16 hilos, frente a un techo teórico estimado de 8.93x (basado en una fracción paralelizable de $P \approx 0.888$). Llegar al 63.2% del máximo teórico es un muy buen resultado. La diferencia restante se debe principalmente a la parte que necesariamente queda serial: las operaciones de I/O (como *save_ppm*) y las fases de fusión de datos.

Uno de los contrastes más llamativos se vio al probar el histograma: la versión con variables locales por hilo fue 299 veces más rápida que la que usaba sección crítica (0.023 s frente a 6.889 s). Esto demuestra de forma muy clara el alto costo de la contención. Con *critical*, los 16 hilos se serializaban cada vez que accedían al mapa compartido, convirtiendo una operación supuestamente paralela en prácticamente secuencial. En cambio, los mapas locales eliminan toda contención durante el conteo y el costo de fusionarlos al final es prácticamente nulo comparado a la ejecución total.

El análisis con *perf* descartó problemas de *false sharing*: la tasa de *cache misses* incluso bajó ligeramente, pasando de 1.35 % con 1 hilo a 1.17 % con 16 hilos. Si hubiera *false sharing*, habríamos visto un aumento claro con más hilos.

Respecto a las configuraciones de afinidad, la más rápida en conjunto fue *closed + threads + 16* (4.588 s). Esto tiene sentido porque al anclar dos hilos por núcleo físico, los datos que carga un hilo permanecen en la caché L1/L2 compartida, beneficiando al compañero del mismo núcleo. Para la Tarea A de forma individual, la mejor configuración fue *spread + cores + 8*, esto demuestra que el acceso a cache y la configuración de acceso y alojamiento de los hilos afecta directamente a su ejecución.

En resumen, el proyecto deja claro que optimizar un programa paralelo va mucho más allá de poner `#pragma omp parallel for`. Requiere analizar cuidadosamente la distribución de trabajo, elegir el scheduler adecuado, diseñar estructuras de datos que eviten contención y *false sharing*, considerar la topología del hardware para fijar los hilos y ajustar las flags de compilación así como la refactorización del código para aprovechar la vectorización SIMD.

Bibliografía

- [1] "OpenMP* Imbalance and Scheduling Overhead," *Intel*, 2023.
<https://www.intel.com/content/www/us/en/docs/vtune-profiler/cookbook/2023-2/openmp-imbalance-and-scheduling-overhead.html> (accedido May 27, 2026).
- [2] JC, "Caches and the problem of false sharing, a primer in C++17," *Medium*, Mar. 07, 2023. https://medium.com/@joao_vaz/caches-and-the-problem-of-false-sharing-a-primer-in-c-17-1177ad07a625 (accedido May 27, 2026).
- [3] "Auto-Vectorization and SPMD - Algorithmica," *Algorithmica.org*, 2021.
<https://en.algorithmica.org/hpc/simd/auto-vectorization/> (accedido May 27, 2026).